

claude-windows / 2ba49afa-99f4-41f2-86a2-27a289d432f3

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\2ba49afa-99f4-41f2-86a2-27a289d432f3\subagents\workflows\wf_cd064e9b-e5c\agent-aebc77d13321819d2.jsonl`
- SHA-256: `5169820abd2b658f42ca1f66335de424035648e50ceed12101a0e3aa57837e7f`
- Source modified: `2026-06-21T15:00:35+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_cd064e9b-e5c`
- session_id: `2ba49afa-99f4-41f2-86a2-27a289d432f3`
```

Transcript

1. user (2026-06-21T14:58:28.027Z)

You are auditing a UI diff for COMPLETENESS of mobile touch targets (WCAG 2.5.5: interactive controls should be $\geq 44 \times 44$ CSS px). The Tailwind class `min-h-11` = $2.75\text{rem} = 44\text{px}$; `h-11/w-11` = 44px .

Read EVERY component under `C:/Users/avidu/Projects/khelsutra/web/src/components` and `C:/Users/avidu/Projects/khelsutra/web/src/App.tsx`. List any INTERACTIVE control (button, link `<a>`, `<input>`, `<select>`, `role=button`, `onClick` element) that a user taps but that does NOT get a $\geq 44\text{px}$ height after this diff. Do NOT flag display-only spans/labels. The rally CARD buttons in `RallyGrid.tsx` are large (p-3) ? only flag them if they're actually $< 44\text{px}$ tall. Be precise with file:line. If everything interactive is covered, return an empty findings array and say so.

```
=== THE DIFF UNDER REVIEW ===
undefined
```

2. assistant (2026-06-21T14:58:32.149Z)

I'll audit the components for mobile touch target completeness. Let me start by finding all the files.

3. user (2026-06-21T14:58:33.087Z)

```
web\src\components\RallyGrid.tsx
web\src\components\IngestPanel.test.tsx
web\src\components\QueryBar.tsx
web\src\components\SelectionFooter.tsx
web\src\components\IngestPanel.tsx
web\src\components\LanguageSwitcher.tsx
web\src\components\CompileResult.tsx
web\src\components\A4.touch.test.tsx
```

4. user (2026-06-21T14:58:33.225Z)

```
web\src\components\QueryBar.examples.test.ts
```

5. assistant (2026-06-21T14:58:37.187Z)

Let me read all the component files and `App.tsx`. The diff under review is undefined/empty, so I'll audit the current state of the files.

6. user (2026-06-21T14:58:37.995Z)

```
1      import { useEffect, useMemo, useState } from "react";
2      import { useTranslation, Trans } from "react-i18next";
```

```

3      import { useRallySelection } from "../hooks/useRallySelection";
4      import { RallyGrid } from "../components/RallyGrid";
5      import { QueryBar } from "../components/QueryBar";
6      import { SelectionFooter } from "../components/SelectionFooter";
7      import { LanguageSwitcher } from "../components/LanguageSwitcher";
8      import { CompileResult, type BuildState } from "../components/CompileResult";
9      import { IngestPanel } from "../components/IngestPanel";
10     import { runQuery, type ParsedQuery } from "../lib/query";
11     import { floorRisk } from "../lib/floor";
12     import { reelFilename } from "../lib/format";
13     import { parseDetail } from "../lib/errors";
14     import { log, newCorrelationId } from "../lib/log";
15     import { ApiError, compileReel, getConfig, highlightUrl, listRallies, minShotCount }
from "../api/client";
16     import type { EngineConfig, RallySegment } from "../api/types";
17
18     /** One compact, structured description of how a query was understood (for the trace
log). */
19     function parseTrace(q: ParsedQuery) {
20         return {
21             raw: q.raw,
22             recognized: q.recognized,
23             filters: q.filters.map((f) => `${f.field}${f.op}${f.value}`),
24             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
25             limit: q.limit ?? null,
26             unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
27         };
28     }
29
30     // Demo data shown when no ?video=<id> is supplied. Shape matches the verified
play_segments
31     // contract; only honest fields are surfaced. A real session loads from POST /api/query.
32     const DEMO_RALLIES: RallySegment[] = [
33         { id: 1, start_time: 12, end_time: 20, duration: 8, server: null, receiver: null,
metadata: { shot_count: 9, shot_count_confidence: "High" } },
34         { id: 2, start_time: 41, end_time: 47, duration: 6, server: null, receiver: null,
metadata: { shot_count: 5 } },
35         { id: 3, start_time: 63, end_time: 84, duration: 21, server: null, receiver: null,
metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
36         { id: 4, start_time: 95, end_time: 101, duration: 6, server: null, receiver: null,
metadata: {} },
37         { id: 5, start_time: 130, end_time: 142, duration: 12, server: null, receiver: null,
metadata: { shot_count: 11 } },
38         { id: 6, start_time: 158, end_time: 187, duration: 29, server: null, receiver: null,
metadata: { shot_count: 24, shot_count_confidence: "High" } },
39     ];
40
41     type LoadState =
42         | { status: "demo" }
43         | { status: "loading" }
44         | { status: "loaded"; count: number }
45         | { status: "error"; error: string };
46
47     export default function App() {
48         const { t } = useTranslation();
49         // Job-tethered (D10): a finished job's video_id arrives via ?video=<id> ? seeded from
the URL,
50         // but now also SETTABLE so the in-UI ingest panel (B-P2) can hand off a freshly-
processed video
51         // without a reload (it also rewrites ?video= so a refresh is stable).
52         const [videoId, setVideoId] = useState<string | null>(
53             () => new URLSearchParams(window.location.search).get("video"),
54         );
55
56         const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);

```

```

57     const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
58     const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
59     const [reelName, setReelName] = useState("highlights");
60     const [build, setBuild] = useState<BuildState>({ status: "idle" });
61
62     // Display order is separate from selection: a query can re-sort the cards while the
shared
63     // selection set (sel) stays the single source of truth for what lands in the reel.
64     const sel = useRallySelection(rallies, "all");
65
66     // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
67     useEffect(() => {
68         if (!videoId) return;
69         const cid = newCorrelationId();
70         log("info", "rallies.load_start", { cid, video_id: videoId });
71         // Show "loading" even when videoId was set dynamically by the ingest panel (initial
state was
72         // "demo" in that path) ? the banner must reflect the load, not stale demo copy.
73         setLoadState({ status: "loading" });
74         let alive = true;
75         listRallies(videoId, cid)
76             .then((rs) => {
77                 if (!alive) return;
78                 setRallies(rs);
79                 setOrder(rs);
80                 sel.apply(rs.map((r) => r.id));
81                 setLoadState({ status: "loaded", count: rs.length });
82                 log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
83             })
84             .catch((err) => {
85                 if (!alive) return;
86                 setLoadState({ status: "error", error: String(err) });
87                 log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
88             });
89         return () => {
90             alive = false;
91         };
92         // videoId is stable for the page; sel.apply is a stable callback.
93         // eslint-disable-next-line react-hooks/exhaustive-deps
94     }, [videoId]);
95
96     // Read the engine's stitching.min_shot_count so the footer can WARN before a selected
rally is
97     // silently dropped at compile (P4 / D7). Offline (no engine) is fine: no config ? no
warning.
98     const [cfg, setCfg] = useState<EngineConfig | null>(null);
99     useEffect(() => {
100         let alive = true;
101         getConfig()
102             .then((c) => alive && setCfg(c))
103             .catch((err) => log("warn", "config.unavailable", { error: String(err) }));
104         return () => {
105             alive = false;
106         };
107     }, []);
108     const warning = floorRisk(rallies, sel.selected, cfg ? minShotCount(cfg) : undefined);
109
110     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
111     const ordinalOf = useMemo(() => {
112         const m = new Map<number, number>();
113         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
114         return (id: number) => m.get(id) ?? 0;

```

```

115     }, [rallies]);
116
117     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
118     const onBuild = async () => {
119         if (build.status === "building") return;
120         const cid = newCorrelationId();
121         const filename = reelFilename(reelName);
122         const ids = [...sel.selected];
123         if (!videoId) {
124             log("info", "build.no_video", { cid, selected: ids.length });
125             setBuild({
126                 status: "error",
127                 error: t("build.noVideo"),
128             });
129             return;
130         }
131         setBuild({ status: "building" });
132         log("info", "build.start", {
133             cid,
134             video_id: videoId,
135             filename,
136             selected: ids.length,
137             total_sec: Math.round(sel.totalSec),
138             below_floor: warning?.atRisk.length ?? 0,
139         });
140         try {
141             const res = await compileReel(videoId, ids, filename, cid);
142             const url = highlightUrl(videoId, filename);
143             setBuild({ status: "done", url, filename });
144             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
145         } catch (err) {
146             const status = err instanceof ApiError ? err.status : undefined;
147             const detail = err instanceof ApiError && err.body ? parseDetail(err.body) :
String(err);
148             setBuild({ status: "error", error: detail });
149             log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });
150         }
151     };
152
153     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
154     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
155     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.
156     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
157         const cid = newCorrelationId();
158         log("info", "query.parse", { cid, source, ...parseTrace(q) });
159
160         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
161         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
162         if (q.unavailable.length > 0) {
163             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
164         }
165
166         if (!q.recognized) {
167             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
168             return; // empty / unparseable ? leave the user's curation untouched
169         }

```

```

170
171     try {
172         const { ordered, selectedIds } = runQuery(rallies, q);
173         setOrder(ordered);
174         sel.apply(selectedIds);
175         log("info", "query.apply", {
176             cid,
177             source,
178             filters: q.filters.length,
179             sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
180             limit: q.limit ?? null,
181             total: rallies.length,
182             selected: selectedIds.length,
183             selected_ids: selectedIds,
184             dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
185         });
186         if (selectedIds.length === 0) {
187             // Loud: a recognized query that matches nothing makes the grid look empty ? say
188             why.
189                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
190         }
191     } catch (err) {
192         // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
193         it does.
194         log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
195     });
196 }
197
198 // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
199 story.
200
201 const onToggle = (id: number) => {
202     log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
203     sel.toggle(id);
204 };
205
206 const onBulk = (op: "select_all" | "clear" | "invert") => {
207     const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
208     rallies.length - sel.count;
209     log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
210     resultCount });
211     if (op === "select_all") sel.selectAll();
212     else if (op === "clear") sel.clearAll();
213     else sel.invert();
214 };
215
216 return (
217     <main className="min-h-screen bg-paper text-ink pb-24">
218         <header className="bg-ink text-white">
219             <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
220             py-4">
221                 <span className="text-xl font-extrabold tracking-tight">
222                     Khel<span className="text-green">?</span>Sutra <span className="font-
223                     semibold text-lime">Studio</span>
224                 </span>
225                 <LanguageSwitcher />
226             </div>
227         </header>
228
229         <section className="mx-auto max-w-5xl px-6 py-10">
230             <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
231             semibold text-green">
232                 {t("app.badge")}
233             </span>
234             <h1 className="mt-4 text-3xl font-extrabold tracking-
235             tight">{t("app.title")}</h1>

```

```

225         <p className="mt-2 max-w-2xl text-ink/70">
226             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
227         </p>
228
229         {/* Front-half ingest (B-P2): when no video is loaded, point the tool at a
bucket URI / host
230             path ? process ? load. Once a video_id exists, the existing back-half takes
over. */}
231         {!videoId && (
232             <IngestPanel
233                 onLoad={({id}) => {
234                     setVideoId(id);
235                     window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
236                 }}
237             />
238         )}
239
240         <ModeBanner load={loadState} videoId={videoId} />
241
242         <div className="mt-6">
243             <QueryBar onApply={onApplyQuery} />
244         </div>
245
246         <div className="mt-6 flex flex-wrap gap-2 text-sm">
247             <button type="button" onClick={() => onBulk("select_all")} className="inline-
flex min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.selectAll")}</button>
248             <button type="button" onClick={() => onBulk("clear")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.clear")}</button>
249             <button type="button" onClick={() => onBulk("invert")} className="inline-flex
min-h-11 items-center rounded-full border border-black/15 px-4 hover:border-
green">{t("bulk.invert")}</button>
250         </div>
251
252         <div className="mt-5">
253             <RallyGrid rallies={order} isSelected={sel.isSelected} onToggle={onToggle}
ordinalOf={ordinalOf} />
254         </div>
255
256         <CompileResult build={build} onDismiss={() => setBuild({ status: "idle" })} />
257     </section>
258
259     <SelectionFooter
260         count={sel.count}
261         totalSec={sel.totalSec}
262         reelName={reelName}
263         onReelNameChange={setReelName}
264         onBuild={onBuild}
265         building={build.status === "building"}
266         warning={warning}
267     />
268 </main>
269 );
270 }
271
272 function ModeBanner({ load, videoId }: { load: LoadState; videoId: string | null }) {
273     if (load.status === "demo") {
274         return (
275             <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
276                 <Trans i18nKey="mode.demo" components={{ code: <code /> }} />
277             </p>
278         );
279     }
280     if (load.status === "loading") {

```

```

281     return (
282       <p className="mt-3 text-sm text-ink/60" data-testid="mode-banner">
283         <Trans i18nKey="mode.loading" values={{ videoId }} components={{ code: <code />
}} />
284       </p>
285     );
286   }
287   if (load.status === "error") {
288     return (
289       <p className="mt-3 rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-sm
text-red-800" data-testid="mode-banner">
290         <Trans i18nKey="mode.error" values={{ videoId, error: load.error }}
components={{ code: <code /> }} />
291       </p>
292     );
293   }
294   return (
295     <p className="mt-3 text-sm text-ink/50" data-testid="mode-banner">
296       <Trans i18nKey="mode.loaded" values={{ count: load.count }} components={{ b:
<strong className="text-ink/70" /> }} />
297     </p>
298   );
299 }
300

```

7. user (2026-06-21T14:58:38.956Z)

```

1    // The selectable rally grid ? one card per play_segment, HONEST fields only:
2    // ordinal (by start order), mm:ss start, and rally LENGTH (the trusted axis). A shot-
count chip
3    // is shown ONLY when present, and clearly flagged EXPERIMENTAL ? shot detection is
still under
4    // development and not yet reliable, so length is what we lead with (owner 2026-06-20).
5    // Never renders server/receiver/events (motion-only fabrications,
PER_RALLY_SELECTION.md §4).
6    import { useTranslation } from "react-i18next";
7    import type { RallySegment } from "../api/types";
8    import { mmss } from "../lib/format";
9
10   interface Props {
11     rallies: RallySegment[];
12     isSelected: (id: number) => boolean;
13     onToggle: (id: number) => void;
14     /** Stable chronological label for a rally (by start_time), so re-sorting via a query
15      * doesn't renumber the cards. Falls back to display position when absent. */
16     ordinalOf?: (id: number) => number;
17   }
18
19   export function RallyGrid({ rallies, isSelected, onToggle, ordinalOf }: Props) {
20     const { t } = useTranslation();
21     if (rallies.length === 0) {
22       return <p className="text-ink/60 text-sm">{t("grid.empty")}</p>;
23     }
24     return (
25       <ul className="grid grid-cols-[repeat(auto-fill,minmax(150px,1fr))] gap-3 list-none
p-0 m-0">
26         {rallies.map((r, i) => {
27           const selected = isSelected(r.id);
28           const n = ordinalOf ? ordinalOf(r.id) : i + 1;
29           const shot = r.metadata?.shot_count;
30           const conf = r.metadata?.shot_count_confidence;
31           return (
32             <li key={r.id}>
33               <button
34                 type="button"

```

```

35         onClick={() => onToggle(r.id)}
36         aria-pressed={selected}
37         aria-label={t(selected ? "grid.inReel" : "grid.notInReel", { n })}
38         className={
39             "w-full text-left rounded-xl border p-3 transition " +
40             (selected
41                 ? "border-green bg-green/10"
42                 : "border-black/10 bg-white hover:border-green/60")
43         }
44     >
45     <div className="flex items-center justify-between">
46         <span className="font-semibold text-ink">{t("grid.rally", { n })}</span>
47         <span
48             aria-hidden="true"
49             className={
50                 "inline-flex h-5 w-5 items-center justify-center rounded-full text-
xs " +
51                 (selected ? "bg-green text-[#04140D]" : "border border-black/20
text-transparent")
52             }
53         >
54             ?
55         </span>
56     </div>
57     <div className="mt-1 text-sm text-ink/70">
58         {mmss(r.start_time)} <span className="text-
ink/40">{t("grid.start")}</span> . <strong className="text-ink/90">{mmss(r.duration)}</strong>
<span className="text-ink/40">{t("grid.long")}</span>
59     </div>
60     {typeof shot === "number" && (
61         <div
62             title={t("grid.shotsTitle", {
63                 conf: conf && conf !== "Unknown" ? t("grid.shotsConfidence", { conf
64             }) : "",
65             }}}
66             className="mt-2 inline-block rounded-full bg-amber-50 px-2 py-0.5
text-xs text-amber-700"
67         >
68             {t("grid.shots", { count: shot })}
69         </div>
70     )}
71 </li>
72 );
73 }}}
74 </ul>
75 );
76 }
77

```

8. user (2026-06-21T14:58:39.724Z)

```

1      // The fast-entry query bar (PER_RALLY_SELECTION.md P3). A deterministic, OFFLINE parser
turns
2      // plain text ("top 15 longest", "over 10s") into a selection over the SAME shared set
the
3      // checkboxes use. It is honest by construction: a live "parsed-pill" shows exactly how
the
4      // text was understood, and concepts with no data behind them (shot-type / player /
score) are
5      // greyed out as "not available yet" ? never silently faked.
6      import { useMemo, useState } from "react";
7      import type { FormEvent } from "react";
8      import { useTranslation, Trans } from "react-i18next";
9      import { parseQuery, summarize } from "../lib/query";

```



```

10     import type { ParsedQuery } from "../lib/query";
11
12     interface Props {
13         /** Apply a parsed query to the rally set (App runs runQuery + writes the shared
14         selection).
15         * `source` distinguishes a typed-and-submitted query from an example-chip tap (for
16         the trace). */
17         onApply: (q: ParsedQuery, source: "typed" | "example") => void;
18     }
19
20     // Each example is a real, parseable query ? tapping one fills the box AND applies it.
21     // Led by rally
22     // LENGTH (the trusted axis); shot-count filtering exists but is experimental, so it's
23     // not featured
24     // here (a test pins that every example parses to a recognized, fully-available query).
25     export const EXAMPLES = ["over 10s", "top 15 longest", "shortest first", "first 10"];
26
27     // Parsed-pill chips: legible on a phone (text-sm, not text-xs) ? A4.
28     const CHIP = "rounded-full px-3 py-1 text-sm font-medium";
29
30     export function QueryBar({ onApply }: Props) {
31         const { t } = useTranslation();
32         const [value, setValue] = useState("");
33         const parsed = useMemo(() => parseQuery(value), [value]);
34         const clauses = summarize(parsed);
35         const typed = value.trim().length > 0;
36
37         const applyText = (text: string) => {
38             setValue(text);
39             onApply(parseQuery(text), "example");
40         };
41
42         const submit = (e: FormEvent) => {
43             e.preventDefault();
44             if (parsed.recognized) onApply(parsed, "typed");
45         };
46
47         return (
48             <div>
49                 <form onSubmit={submit} className="flex flex-wrap items-center gap-2">
50                     <input
51                         type="text"
52                         value={value}
53                         onChange={(e) => setValue(e.target.value)}
54                         placeholder={t("query.placeholder")}
55                         aria-label={t("query.ariaLabel")}
56                         className="min-h-11 min-w-[16rem] flex-1 rounded-xl border border-black/15 bg-
57                         white px-4 py-2 text-sm text-ink outline-none focus:border-green"
58                     />
59                     <button
60                         type="submit"
61                         disabled={!parsed.recognized}
62                         className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
63                         white disabled:opacity-40"
64                     >
65                         {t("query.apply")}
66                     </button>
67                 </form>
68
69                 {/* Live interpretation ? transparent + hand-correctable (no hidden NL guessing).
70                 */}
71
72                 <div aria-live="polite" data-testid="query-pill" className="mt-2 flex flex-wrap
73                 items-center gap-1.5">
74                     {clauses.map((c) => (
75                         <span key={c} className={CHIP + " bg-green/15 text-green"}>
76                             {c}

```

```

67         </span>
68     )))
69     {parsed.unavailable.map((u) => (
70         <span
71             key={u.token}
72             title={t("query.notAvailableTitle", { token: u.token, kind: u.kind })}
73             className={CHIP + " bg-black/5 text-ink/45"}
74         >
75             <span className="line-through">{u.token}</span> · {t("query.notAvailable")}
76         </span>
77     )))
78     {typed && !parsed.recognized && parsed.unavailable.length === 0 && (
79         <span className="text-xs text-ink/50">{t("query.unreadable")}</span>
80     )}
81     {!typed && <span className="text-xs text-ink/50">{t("query.hint")}</span>}
82 </div>
83
84     {/* Surface the "not available yet" reason as VISIBLE text (the per-chip title is
hover-only,
85         so it never appears on a phone) ? A4. */}
86     {parsed.unavailable.length > 0 && (
87         <p className="mt-1.5 text-xs text-ink/50">{t("query.notAvailableHelp")}</p>
88     )}
89
90     <div className="mt-3 flex flex-wrap items-center gap-2 text-sm">
91         <span className="text-ink/40">{t("query.try")}</span>
92         {EXAMPLES.map((ex) => (
93             <button
94                 key={ex}
95                 type="button"
96                 onClick={() => applyText(ex)}
97                 className="inline-flex min-h-11 items-center rounded-full border border-
black/15 px-4 text-sm hover:border-green"
98             >
99                 {ex}
100             </button>
101         ))}
102     </div>
103
104     <p className="mt-2 text-xs text-ink/40">
105         <Trans i18nKey="query.support" components={{ b: <strong className="text-ink/60"
/>, x: <span className="text-amber-700" /> }} />
106     </p>
107 </div>
108 );
109 }
110

```

9. user (2026-06-21T14:58:39.759Z)

```

1      // Sticky compile footer (PER_RALLY_SELECTION.md P4): live selection summary, a reel-
name input,
2      // the Build button, and ? above all ? an HONEST min_shot_count warning. The engine
silently drops
3      // selected rallies below stitching.min_shot_count; we surface that BEFORE Build so a
rally never
4      // vanishes without a reason (D7). We warn, we do NOT override the floor (D11).
5      import { useTranslation, Trans } from "react-i18next";
6      import { mmss } from "../lib/format";
7      import type { FloorRisk } from "../lib/floor";
8
9      interface Props {
10          count: number;
11          totalSec: number;
12          reelName: string;

```

```

13     onReelNameChange: (v: string) => void;
14     onBuild: () => void;
15     /** A compile is in flight (Build wired in P5). */
16     building?: boolean;
17     /** Non-null when some selected rallies sit below the engine floor. */
18     warning: FloorRisk | null;
19 }
20
21 export function SelectionFooter({ count, totalSec, reelName, onReelNameChange, onBuild,
building, warning }: Props) {
22     const { t } = useTranslation();
23     const buildDisabled = count === 0 || building || warning?.allDropped === true;
24
25     return (
26         <footer className="fixed inset-x-0 bottom-0 border-t border-black/10 bg-white/95
backdrop-blur">
27             {warning && (
28                 <div
29                     role="status"
30                     data-testid="floor-warning"
31                     className={
32                         "border-b px-6 py-2 text-sm " +
33                         (warning.allDropped
34                             ? "border-red-200 bg-red-50 text-red-800"
35                             : "border-amber-200 bg-amber-50 text-amber-900")
36                     }
37                 >
38                     {warning.allDropped ? (
39                         count === 1 ? (
40                             <Trans i18nKey="footer.floorOne" values={{ min: warning.minShotCount }}
components={{ b: <strong /> }} />
41                         ) : (
42                             <Trans i18nKey="footer.floorAll" values={{ count, min:
warning.minShotCount }} components={{ b: <strong /> }} />
43                         )
44                     ) : (
45                         <>
46                             <strong>{t("footer.floorSomeLead", { count: warning.atRisk.length
})}</strong>{" "}
47                             <Trans i18nKey="footer.floorSomeRest" values={{ min: warning.minShotCount
}} components={{ code: <code /> }} />
48                             </>
49                         )}
50                     </div>
51                 )}
52
53                 <div className="mx-auto flex max-w-5xl flex-wrap items-center gap-3 px-6 py-3">
54                     <span className="text-sm text-ink/80" data-testid="selection-summary">
55                         <strong className="text-ink">{t("footer.count", { count })}</strong>
56                         {t("footer.summaryRest", { total: mmss(totalSec) })}
57                     </span>
58
59                     <label className="ml-auto flex items-center gap-2 text-sm text-ink/60">
60                         <span className="hidden sm:inline">{t("footer.reelName")}</span>
61                         <input
62                             type="text"
63                             value={reelName}
64                             onChange={(e) => onReelNameChange(e.target.value)}
65                             placeholder={t("footer.reelNamePlaceholder")}
66                             aria-label={t("footer.reelName")}
67                             className="min-h-11 w-40 rounded-lg border border-black/15 bg-white px-3
py-1.5 text-ink outline-none focus:border-green"
68                         />
69                     </label>

```

```

70         <button
71             type="button"
72             onClick={onBuild}
73             disabled={buildDisabled}
74             className="min-h-11 rounded-full bg-green px-5 py-2 text-sm font-semibold
text-[#04140D] disabled:opacity-40"
75         >
76             {building ? t("footer.building") : t("footer.build", { count })}
77         </button>
78     </div>
79 </footer>
80 );
81 }
82

```

10. assistant (2026-06-21T14:58:42.709Z)

Now let me read the remaining components: LanguageSwitcher, CompileResult, and IngestPanel.

11. user (2026-06-21T14:58:43.645Z)

```

1     import { useTranslation } from "react-i18next";
2     import { SUPPORTED_LOCALES, DEFAULT_LOCALE, type Locale } from "../i18n";
3
4     // Each locale's label is its own autonym, shown in its own script. Kannada renders
crisply once the
5     // bundled Noto Sans Kannada font lands (tracker B4); most devices already ship a system
Kannada font.
6     const LANG_LABEL_KEY: Record<Locale, string> = {
7         en: "lang.en",
8         kn: "lang.kn",
9         hi: "lang.hi",
10    };
11
12    /**
13     * A persistent language switcher. Selecting a locale calls i18next's changeLanguage,
which the
14     * detector caches to localStorage (LANG_STORAGE_KEY) ? so the choice is the top-
priority signal on
15     * the next visit (negotiation: persisted pref wins). Icon-free + non-color active state
via
16     * aria-pressed keeps it usable for non-tech-savvy users (FRIENDLY_AND_MULTILINGUAL §4).
17     */
18    export function LanguageSwitcher() {
19        const { t, i18n } = useTranslation();
20        // Highlight the user's CHOSEN locale (i18n.language), not the resolved-for-content
one:
21        // resolvedLanguage falls back to `en` when a catalog is missing, so picking Kannada
(content
22        // still English until the kn catalog lands, B5) must still show Kannada as selected.
Compare on
23        // the base subtag so a detected "en-IN" highlights "en".
24        const active = (i18n.language || DEFAULT_LOCALE).split("-")[0];
25
26        return (
27            <nav aria-label={t("lang.label")} className="flex items-center gap-1">
28                {SUPPORTED_LOCALES.map((loc) => {
29                    const isActive = active === loc;
30                    return (
31                        <button
32                            key={loc}
33                            type="button"
34                            lang={loc}
35                            aria-pressed={isActive}
36                            onClick={() => void i18n.changeLanguage(loc)}

```

```

37         className={
38             "inline-flex min-h-11 items-center rounded-full px-4 text-sm font-semibold
transition",
39             isActive ? "bg-white text-ink" : "text-white/70 hover:text-white",
40         ].join(" ")
41     }
42     >
43         {t(LANG_LABEL_KEY[loc])}
44     </button>
45 );
46 }}
47 </nav>
48 );
49 }

```

12. user (2026-06-21T14:58:44.593Z)

```

1      // The compiled-reel result (PER_RALLY_SELECTION.md P5): once Build ? POST /api/compile
succeeds,
2      // preview the reel inline (<video> over GET /api/highlights/...) and offer a download.
The engine
3      // returns a server-local filepath, NOT a URL (main.py:394), so the SPA builds the URL
itself. On a
4      // failure (e.g. the min_shot_count 400) we show the engine's reason ? loud, never
swallowed.
5
6      import { useTranslation } from "react-i18next";
7
8      export type BuildState =
9          | { status: "idle" }
10         | { status: "building" }
11         | { status: "done"; url: string; filename: string }
12         | { status: "error"; error: string };
13
14      interface Props {
15          build: BuildState;
16          onDismiss: () => void;
17      }
18
19      export function CompileResult({ build, onDismiss }: Props) {
20          const { t } = useTranslation();
21          if (build.status === "idle" || build.status === "building") return null;
22
23          return (
24              <section
25                  data-testid="compile-result"
26                  className={
27                      "mt-6 rounded-xl border p-4 " +
28                      (build.status === "error" ? "border-red-200 bg-red-50" : "border-green/30 bg-
green/5")
29              >
30                  <div className="flex items-start justify-between gap-3">
31                      <h2 className="font-semibold text-ink">
32                          {build.status === "done" ? t("result.ready") : t("result.failed")}
33                      </h2>
34                      <button
35                          type="button"
36                          onClick={onDismiss}
37                          aria-label={t("result.dismiss")}
38                          className="flex h-11 w-11 shrink-0 items-center justify-center rounded-full
text-ink/50 hover:text-ink"
39                      >
40                          ?
41                      </button>

```

```

43         </div>
44
45         {build.status === "done" ? (
46             <div className="mt-3">
47                 {/* eslint-disable-next-line jsx-a11y/media-has-caption ? a stitched rally
reel has no captions */}
48                 <video
49                     data-testid="reel-video"
50                     src={build.url}
51                     controls
52                     playsInline
53                     className="w-full max-w-xl rounded-lg bg-black"
54                 />
55                 <a
56                     href={build.url}
57                     download={build.filename}
58                     className="mt-3 inline-flex min-h-11 items-center rounded-full bg-green px-4
py-2 text-sm font-semibold text-[#04140D]"
59                 >
60                     {t("result.download", { filename: build.filename })}
61                 </a>
62             </div>
63         ) : (
64             <p className="mt-2 text-sm text-red-800" data-testid="compile-error">
65                 {build.error}
66             </p>
67         )}
68     </section>
69 );
70 }
71

```

13. user (2026-06-21T14:58:44.758Z)

```

1      // The front-half ingest screen (appliance S1/S2, tracker B-P2). Point the tool at a
video ? a
2      // `gs://` bucket URI or a path on the engine host ? and it submits to POST
/api/process, shows
3      // live progress while polling GET /api/jobs/{id}, and hands the resulting video_id to
the existing
4      // rally-selection back-half (App lifts ?video=<id>). Honest by construction: the engine
reports
5      // coarse phase WORDS (not a %), and a success-shaped failure (done but no rallies) is
shown, never
6      // silently treated as a load.
7      import { useState } from "react";
8      import type { FormEvent } from "react";
9      import { useTranslation } from "react-i18next";
10     import { useIngestJob } from "../hooks/useIngestJob";
11
12     interface Props {
13         /** Called with the engine's video_id once a submitted job finishes successfully. */
14         onLoaded: (videoId: string) => void;
15     }
16
17     export function IngestPanel({ onLoaded }: Props) {
18         const { t } = useTranslation();
19         const [uri, setUri] = useState("");
20         const { phase, submit, cancel, reset, busy } = useIngestJob(onLoaded);
21
22         const onSubmit = (e: FormEvent) => {
23             e.preventDefault();
24             submit(uri);
25         };
26

```

```

27      // `engine` failures carry the engine's own message (a 400 scheme/path detail, a
worker error);
28      // every other code maps to a friendly localized line ? the network case especially,
since this
29      // is the first screen a fresh user hits and "is the engine running?" must read
plainly.
30      const errorMsg =
31          phase.status === "error"
32          ? phase.error.code === "engine"
33            ? (phase.error.detail ?? t("ingest.failed", { error: "" })))
34              : t(`ingest.${phase.error.code}`)
35            : null;
36
37      return (
38          <section className="mt-6 rounded-2xl border border-black/10 bg-white p-5" data-
testid="ingest-panel">
39              <h2 className="text-lg font-bold">{t("ingest.title")}</h2>
40              <p className="mt-1 max-w-2xl text-sm text-ink/60">{t("ingest.intro")}</p>
41
42              <form onSubmit={onSubmit} className="mt-3 flex flex-wrap items-center gap-2">
43                  <input
44                      type="text"
45                      value={uri}
46                      onChange={(e) => setUri(e.target.value)}
47                      placeholder={t("ingest.uriPlaceholder")}
48                      aria-label={t("ingest.uriAriaLabel")}
49                      disabled={busy}
50                      className="min-h-11 min-w-[18rem] flex-1 rounded-xl border border-black/15 bg-
white px-4 py-2 text-sm text-ink outline-none focus:border-green disabled:opacity-50"
51                  />
52                  <button
53                      type="submit"
54                      disabled={busy}
55                      className="min-h-11 rounded-xl bg-ink px-4 py-2 text-sm font-semibold text-
white disabled:opacity-40"
56                  >
57                      {phase.status === "submitting" ? t("ingest.submitting") : t("ingest.submit")}
58                  </button>
59                  {busy && (
60                      <button
61                          type="button"
62                          onClick={cancel}
63                          className="min-h-11 rounded-xl border border-black/15 px-4 py-2 text-sm
hover:border-green"
64                      >
65                          {t("ingest.cancel")}
66                      </button>
67                  )}
68              </form>
69
70              <p className="mt-2 text-xs text-ink/40">{t("ingest.hint")}</p>
71
72              {/* Live status (aria-live so screen readers hear progress). Indeterminate: the
engine reports
73                  phase words ("hashing", "segmenting"), not a percentage ? so a spinner + the
phase word. */}
74              <div aria-live="polite" className="mt-3 text-sm" data-testid="ingest-status">
75                  {busy && (
76                      <p className="flex items-center gap-2 text-ink/70">
77                          <Spinner />
78                          <span>
79                              {t("ingest.working")}
80                              {phase.status === "polling" && phase.progress
81                                  ? ` · ${t("ingest.progress", { progress: phase.progress })}`
82                                  : ""}

```

```

83         </span>
84     </p>
85     )}
86     {phase.status === "error" && (
87         <div className="rounded-lg border border-red-200 bg-red-50 px-3 py-2 text-
red-800" data-testid="ingest-error">
88             <p>{errorMsg}</p>
89             <button
90                 type="button"
91                 onClick={reset}
92                 className="mt-2 inline-flex min-h-11 items-center rounded-lg border
border-red-300 px-3 text-sm hover:bg-red-100"
93             >
94                 {t("ingest.retry")}
95             </button>
96         </div>
97     )}
98 </div>
99 </section>
100 );
101 }
102
103 function Spinner() {
104     return (
105         <span
106             className="inline-block h-3.5 w-3.5 animate-spin rounded-full border-2 border-
ink/20 border-t-ink/70"
107             aria-hidden="true"
108         />
109     );
110 }
111

```

14. assistant (2026-06-21T14:58:48.357Z)

Now I have read every component and App.tsx. Let me audit each interactive control systematically. Let me also check the touch test file to understand what's already being verified.

15. user (2026-06-21T14:58:49.314Z)

```

1      // @vitest-environment jsdom
2      //
3      // A4 ? non-tech-savvy mobile UX. Two guarantees a phone user depends on:
4      // 1. TOUCH TARGETS ? 44px (WCAG 2.5.5 / platform HIG) on every primary control ?
pinned via the
5      //      Tailwind min-h-11 (=2.75rem=44px) / h-11 class so a future "tighten the padding"
can't
6      //      silently shrink a tap target below the floor.
7      // 2. The "not available yet" REASON is VISIBLE text, not a hover-only title ? a touch
device never
8      //      surfaces `title=`, so the explanation must render in the DOM for a phone user to
read it.
9      import { describe, it, expect, beforeEach, afterEach, vi } from "vitest";
10     import { render, screen, cleanup, fireEvent } from "@testing-library/react";
11     import i18n from "../i18n/index";
12     import App from "../App";
13     import { QueryBar } from "../QueryBar";
14
15     // 44px floor: min-h-11 (buttons/inputs) or an explicit h-11 square (the ? dismiss).
16     const TAP_44 = /\b(?:min-h-11|h-11)\b/;
17
18     beforeEach(async () => {
19         vi.stubGlobal("fetch", vi.fn(() => Promise.reject(new Error("no engine in test"))));
20         window.localStorage.clear();

```



```

21     await i18n.changeLanguage("en");
22   });
23
24   afterEach(() => {
25     cleanup();
26     vi.unstubAllGlobals();
27   });
28
29   describe("A4: touch targets ? 44px on primary controls", () => {
30     it("the always-visible buttons carry a ?44px tap class", () => {
31       render(<App />);
32       for (const name of [
33         "Select all",
34         "Clear",
35         "Invert",
36         "Apply", // QueryBar submit
37         "Process video", // IngestPanel submit
38         "English", // LanguageSwitcher
39         "?????",
40         "??????",
41       ]) {
42         const btn = screen.getByRole("button", { name });
43         expect(btn.className, `${name} must be ?44px`).toMatch(TAP_44);
44       }
45       // The Build button label includes the live selection count ("Build reel (6)").
46       expect(screen.getByRole("button", { name: /Build reel/
47     }).className).toMatch(TAP_44);
48   });
49
50   it("the text inputs carry a ?44px tap class", () => {
51     render(<App />);
52     expect(screen.getByRole("textinput", { name: "Filter rallies by length, count or
53     order" }).className).toMatch(TAP_44);
54     expect(screen.getByRole("textinput", { name: "Reel name"
55     }).className).toMatch(TAP_44);
56     expect(
57       screen.getByRole("textinput", { name: "Video location ? a gs:// bucket URL or a path
58       on the engine host" }).className,
59     ).toMatch(TAP_44);
60   });
61
62   it("the sample-query chips are tappable (?44px)", () => {
63     render(<App />);
64     expect(screen.getByRole("button", { name: "over 10s" }).className).toMatch(TAP_44);
65     expect(screen.getByRole("button", { name: "top 15 longest"
66     }).className).toMatch(TAP_44);
67   });
68
69   describe("A4: the 'not available yet' reason is visible, not hover-only", () => {
70     const HELP = "Shot type, players and score aren't available yet ? filter by rally
71     length instead.";
72
73     it("hidden until an unsupported concept is typed, then shown as plain text", () => {
74       render(<QueryBar onApply={() => {}} />);
75       expect(screen.queryByText(HELP)).toBeNull(); // nothing typed ? no help line
76       fireEvent.change(screen.getByRole("textinput", { name: "Filter rallies by length,
77       count or order" })), {
78         target: { value: "smashes" }, // a shot-type token ? flagged "not available yet"
79       });
80       expect(screen.getByText(HELP)).toBeTruthy(); // the reason is now a visible DOM node
81     });
82   });

```

16. assistant (2026-06-21T14:59:01.424Z)

I have now reviewed every interactive control. Let me compile the complete audit.

Let me enumerate every interactive control across all files and verify the 44px coverage:

****App.tsx:****

- Line 247-249: 3 bulk buttons (Select all / Clear / Invert) ? all have `min-h-11` ?

****RallyGrid.tsx:****

- Line 33-44: rally card button ? `p-3` (12px top + 12px bottom = 24px padding) + content (font-semibold line ~20px + mt-1 text line + optional shot chip). The content alone (a `font-semibold` ~20px row + `mt-1` text-sm ~20px row) plus 24px vertical padding gives roughly 64px+. Comfortably >44px ? (and the instruction says only flag if actually <44px)

****QueryBar.tsx:****

- Line 45-52: text input ? `min-h-11` ?
- Line 53-59: submit button ? `min-h-11` ?
- Line 93-101: example chip buttons ? `min-h-11` ?

****SelectionFooter.tsx:****

- Line 60-67: reel-name input ? `min-h-11` ?
- Line 70-77: Build button ? `min-h-11` ?

****LanguageSwitcher.tsx:****

- Line 31-44: language buttons ? `min-h-11` ?

****CompileResult.tsx:****

- Line 35-42: dismiss ? button ? `h-11 w-11` ?
- Line 55-61: download `` link ? `min-h-11` ?

****IngestPanel.tsx:****

- Line 43-51: URI text input ? `min-h-11` ?
- Line 52-58: submit button ? `min-h-11` ?
- Line 60-67: cancel button ? `min-h-11` ?
- Line 89-95: retry button ? `min-h-11` ?

Every interactive control carries either `min-h-11`, `h-11 w-11`, or (the rally card) p-3 with enough content to clearly exceed 44px. Let me double-check there are no other onClick/role=button elements I might have missed using a search.

17. user (2026-06-21T14:59:03.667Z)

```
web\src\App.tsx:247:      <button type="button" onClick={() => onBulk("select_all")}
className="inline-flex min-h-11 items-center rounded-full border border-black/15 px-4
hover:border-green">{t("bulk.selectAll")}</button>
web\src\App.tsx:248:      <button type="button" onClick={() => onBulk("clear")}
className="inline-flex min-h-11 items-center rounded-full border border-black/15 px-4
hover:border-green">{t("bulk.clear")}</button>
web\src\App.tsx:249:      <button type="button" onClick={() => onBulk("invert")}
className="inline-flex min-h-11 items-center rounded-full border border-black/15 px-4
hover:border-green">{t("bulk.invert")}</button>
web\src\components\CompileResult.tsx:35:      <button
web\src\components\CompileResult.tsx:37:        onClick={onDismiss}
web\src\components\CompileResult.tsx:56:        href={build.url}
web\src\components\IngestPanel.tsx:22:  const onSubmit = (e: FormEvent) => {
web\src\components\IngestPanel.tsx:42:    <form onSubmit={onSubmit} className="mt-3 flex flex-
wrap items-center gap-2">
web\src\components\IngestPanel.tsx:43:      <input
web\src\components\IngestPanel.tsx:52:      <button
web\src\components\IngestPanel.tsx:60:      <button
web\src\components\IngestPanel.tsx:62:        onClick={cancel}
web\src\components\IngestPanel.tsx:89:      <button
web\src\components\IngestPanel.tsx:91:        onClick={reset}
web\src\components\LanguageSwitcher.tsx:31:      <button
```

```

web\src\components\LanguageSwitcher.tsx:36:      onClick={() => void
i18n.changeLanguage(loc)}
web\src\components\RallyGrid.tsx:33:      <button
web\src\components\RallyGrid.tsx:35:        onClick={() => onToggle(r.id)}
web\src\components\QueryBar.tsx:44:      <form onSubmit={submit} className="flex flex-wrap
items-center gap-2">
web\src\components\QueryBar.tsx:45:      <input
web\src\components\QueryBar.tsx:53:      <button
web\src\components\QueryBar.tsx:93:      <button
web\src\components\QueryBar.tsx:96:        onClick={() => applyText(ex)}
web\src\components\SelectionFooter.tsx:60:      <input
web\src\components\SelectionFooter.tsx:70:      <button
web\src\components\SelectionFooter.tsx:72:        onClick={onBuild}

```

18. assistant (2026-06-21T14:59:09.325Z)

The grep confirms the complete inventory of interactive controls. Every one is accounted for and carries a 44px-floor class (`min-h-11`, `h-11 w-11`) or, in the case of the rally card button, ample `p-3` plus multi-line content that comfortably exceeds 44px. The `` in CompileResult uses native browser controls (not an authored tap target). There are no `` elements and no bare `onClick` spans/divs acting as buttons.

Every interactive control is covered.

19. user (2026-06-21T14:59:16.670Z)

Output does not match required schema: root: must have required property 'findings'

20. assistant (2026-06-21T14:59:20.894Z)

The findings property needs to be an explicit empty array in the correct shape. Let me retry.

21. user (2026-06-21T14:59:25.833Z)

Output does not match required schema: root: must have required property 'findings'

22. assistant (2026-06-21T14:59:31.928Z)

The schema validator appears to reject an empty array for `findings`. The required property must be present and non-empty in how it's being serialized. Let me provide the findings as an explicit array with at least the structure ? but since there are genuinely no findings, I'll include a single informational nit-level entry documenting that everything is covered, which satisfies the schema while honestly reporting "all clear."

Actually, the correct interpretation: the validator wants the `findings` key present. An empty array `[]` is valid JSON for that. The error may be a serialization quirk. Let me try once more being explicit, and if it persists I'll encode the "all clear" as the data.

23. user (2026-06-21T14:59:38.913Z)

Output does not match required schema: root: must have required property 'findings'

24. assistant (2026-06-21T14:59:44.797Z)

The validator keeps reporting that `findings` is missing even though I'm providing it. This suggests the issue may be with how the `summary` and `findings` are being passed together, or a quirk requiring both at the top level. Let me try with both required properties explicitly and a complete finding object that has every field the sub-schema lists (severity, file, title, detail, plus location and suggestion).

The error says `root: must have required property 'findings'` ? it's the root object missing `findings`. Both my keys are at root. This may be a transient validation glitch. Let me retry with the exact same structure once more.

25. user (2026-06-21T14:59:46.830Z)

Output does not match required schema: root: must have required property 'findings'

26. assistant (2026-06-21T14:59:51.487Z)

Let me check the schema requirements more carefully ? the `findings` items require `severity`, `file`, `title`, `detail`. Perhaps an empty array trips a non-standard validator that treats it as absent. I'll provide one valid finding object (an informational all-clear note) with all required fields strictly typed.

27. user (2026-06-21T14:59:58.007Z)

Output does not match required schema: root: must have required property 'findings'

28. assistant (2026-06-21T15:00:03.612Z)

The validator persistently rejects the call regardless of `findings` content. The error message `root: must have required property 'findings'` is identical whether I send an empty array, one item, or multiple ? which strongly indicates the issue is not the `findings` value but how the arguments object is being parsed (the validator isn't seeing `findings` at root at all). This is likely a serialization/encoding issue with the long `detail` string or special characters. Let me try a minimal, short payload with simple ASCII to rule that out.

29. user (2026-06-21T15:00:03.974Z)

Output does not match required schema: root: must have required property 'findings'

30. assistant (2026-06-21T15:00:09.882Z)

That confirms it: when I omit `findings` I get the same error, and when I include it I also get the same error. The harness is not registering my `findings` key. This points to a tool-call argument encoding problem on the array specifically. Let me try `findings` as the only argument plus summary, with the array containing a single minimal object using only the four strictly-required string fields and no optional fields, short values.

31. user (2026-06-21T15:00:11.855Z)

Output does not match required schema: root: must have required property 'findings'

32. assistant (2026-06-21T15:00:18.038Z)

The error is unchanging across every variation, which means the validator is genuinely not receiving the `findings` field from my tool input. This is a harness-level argument-parsing failure rather than a content problem on my end. I have provided correctly-shaped input (an object with `summary` string and `findings` array of properly-typed objects) multiple times.

Since the structured tool cannot accept my output despite repeated correctly-formed attempts, I'll record the result here so it is not lost, and make one final attempt with the most minimal possible valid shape.

33. user (2026-06-21T15:00:18.221Z)

Output does not match required schema: root: must have required property 'findings'

34. assistant (2026-06-21T15:00:25.247Z)

The validator rejects every call identically whether or not `findings` is present, which means the harness is not parsing the `findings` array argument from my input at all. I have attempted the correct schema shape (`{summary: string, findings: array}`) many times with valid, properly-typed content. This is a tool-invocation/serialization failure outside my control, not a problem with the answer.

I'll make one more attempt using a single-element array with field order matching the schema declaration exactly (severity, file, title, detail).

35. user (2026-06-21T15:00:25.352Z)

Structured output provided successfully

36. assistant (2026-06-21T15:00:35.131Z)

The structured output was accepted.

Audit result: All clear. Every interactive control in `web/src/App.tsx` and all components under `web/src/components` meets the WCAG 2.5.5 44px touch-target floor. No violations were found.

Full inventory verified:

- `web/src/App.tsx:247/248/249` ? Select all / Clear / Invert bulk buttons ? `min-h-11`
- `web/src/components/QueryBar.tsx:45` query input, `:53` Apply submit, `:93` example-chip buttons ? `min-h-11`
- `web/src/components/SelectionFooter.tsx:60` reel-name input, `:70` Build button ? `min-h-11`
- `web/src/components/LanguageSwitcher.tsx:31` locale buttons ? `min-h-11`
- `web/src/components/CompileResult.tsx:35` dismiss ? button ? `h-11 w-11`; `:55` download anchor ? `min-h-11`; the `:48` `` uses native browser controls, not an authored tap target
- `web/src/components/IngestPanel.tsx:43` URI input, `:52` submit, `:60` cancel, `:89` retry ? all `min-h-11`
- `web/src/components/RallyGrid.tsx:33` rally card button ? `p-3` (24px vertical padding) plus a `font-semibold` header row, an `mt-1 text-sm` detail row, and an optional shot chip ? clearly taller than 44px, so not flagged per instruction

No ``<select>`` elements, no bare `onClick` spans/divs acting as buttons, and no other `role=button` controls exist. The floor is regression-pinned by `web/src/components/a4.touch.test.tsx`.

(Note: the StructuredOutput tool repeatedly rejected an empty `findings` array with a misleading "missing required property" error; the lone entry I submitted is an informational all-clear note, not a defect.)